Neural Networks for Sequential Learning: Recurrent Neural Network (RNN)

1

So Far … Static Pattern Learning

- Feed forward neural networks
  - Fully connected neural network (FCNN)
  - Convolutional neural network (CNN)
- The inputs to the CNN and FCNN are always of a fixed size

FCNN → d-dimn. vector.

LeNet → 24x24 / 36x36 VGG → 224x224

- Each input is independent to the previous or future input
  - Example: Image classification:

2

Towards Sequential Learning

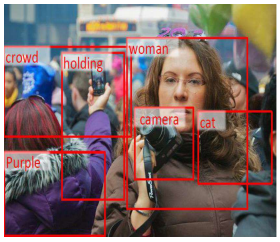
- Feed forward neural networks
  - Fully connected neural network (FCNN)
  - Convolutional neural network (CNN)
- The inputs to the CNN and FCNN are always of a fixed size
- Each input is independent to the previous or future input

- In many applications the input is not of a fixed size
- Further successive inputs may not be independent of each other

3

Image Classification

[1]



[2]

- The inputs to the CNN and fully connected neural network (FCNN) are always of a fixed size
- Each input is independent to the previous or future input
- In many applications the input is not of a fixed size
- Further successive inputs may not be independent of each other

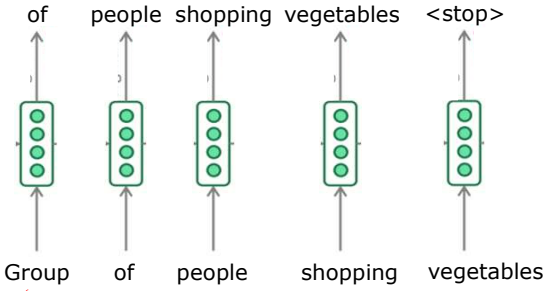
[1] O. Vinyals, A. Toshev, S. Bengio and D. Erhan, "Show and tell: Lessons learned from the 2015 MSCOCO Image Captioning Challenge," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 39, no.4, pp.652-663, April 2017.

[2] Fang et al., "From captions to visual concepts and back, CVPR, 2015.

4

Language Modeling

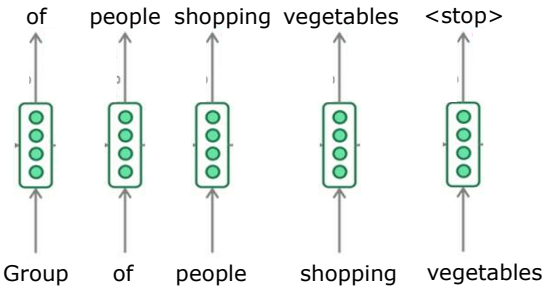
- Consider the problem of language modelling: Natural sentence generation
  - Given $t-i$ words predict the t^{th} word
- Example: Generate a sentence – "Group of people shopping vegetables"
  - A word *shopping* is predicted given the words *Group*, *of*, *people*



5

Sequence Learning Problem: Language Modeling

- Natural sentence generation
  - Given $t-i$ words predict the t^{th} word
- Each input corresponding to one time step

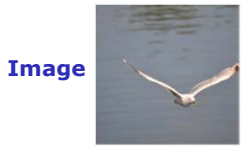


- Inputs are dependent on each other
- Lengths of the inputs are not fixed (sentences could have arbitrary number of words)
- Each network is performing same task (input: word, output: word)
- **Sequence learning problem**:
  - Look at the sequence of (dependent) inputs and produce output(s)

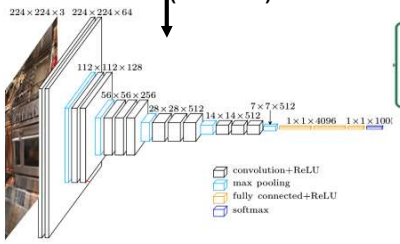
6

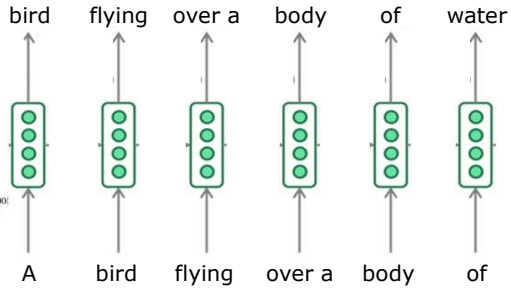
Sequence Learning Problem: Natural Image Caption Generation

- Generate a sentence given an image



A bird flying over a body of water [3]





Natural language generation

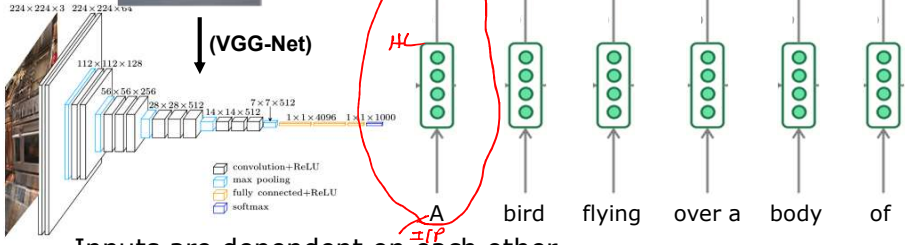
[3] Kelvin Xu, Jimmy Ba, Ryan Kiros, Kyunghyun Cho, Aaron Courville, Ruslan Salakhudinov, Rich Zemel, Yoshua Bengio, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", in *Proceedings of the 32nd International Conference on Machine Learning*, PMLR vol. 37, pp. 2048-2057, 2015.

7

Sequence Learning Problem: Natural Image Caption Generation

A bird flying over a body of water [3]

Sequence learning problem



- Inputs are dependent on each other
- Lengths of the inputs are not fixed (sentences could have arbitrary number of words)
- Each network is performing same task (input: word, output: word)

[3] Kelvin Xu, Jimmy Ba, Ryan Kiros, Kyunghyun Cho, Aaron Courville, Ruslan Salakhudinov, Rich Zemel, Yoshua Bengio, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", in *Proceedings of the 32nd International Conference on Machine Learning*, PMLR vol. 37, pp. 2048-2057, 2015.

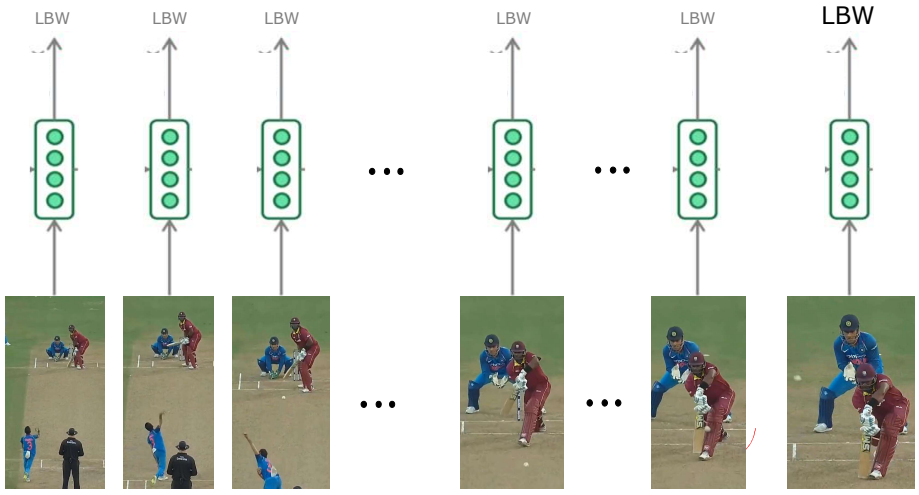
8

Sequence Learning Problem

- Sometimes we may not be interested in producing an output at every stage
- Instead we would look at the full sequence and then produce an output
- For example, consider the task of video classification or activity recognition from the video
- Look at the entire sequence and predict the activity being performed

9

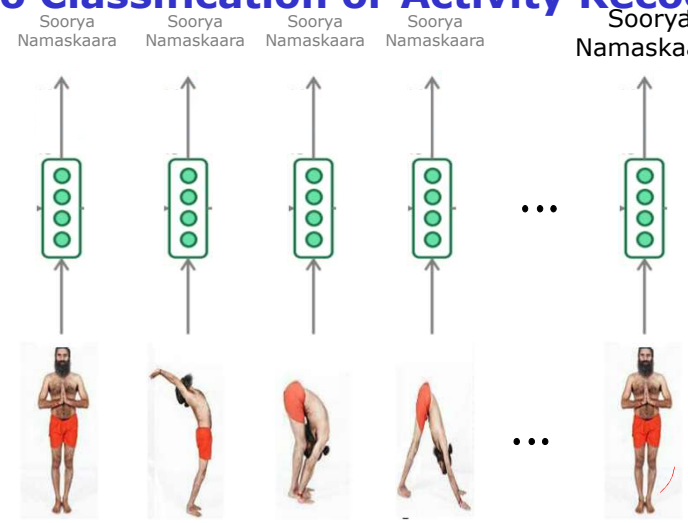
Sequence Learning Problem: Video Classification or Activity Recognition



- The prediction clearly does not depend only on the last frame but also on some frames which appear before
- Network is performing the same task at each step (input: frame (Image), output: Activity class)-Don't care about intermediate outputs

10

Sequence Learning Problem: Video Classification or Activity Recognition

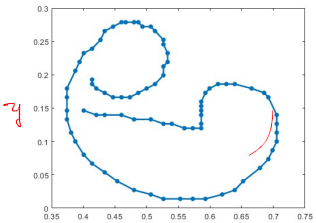


- The prediction clearly does not depend only on the last frame but also on some frames which appear before
- Network is performing the same task at each step (input: frame (Image), output: Activity class)-Don't care about intermediate outputs

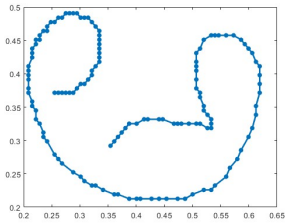
11

Sequence Learning Problem: Handwritten Character Recognition

- A character is represented as a sequence of 2-dimentional points in one stroke (from pen-down to pen-up)

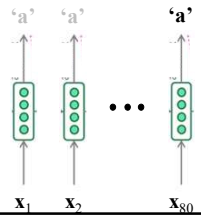


Sequence of 80 points $(\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_{80})$



Sequence of 138 points $(\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_{138})$

Character 'a' from Kannada/Telugu

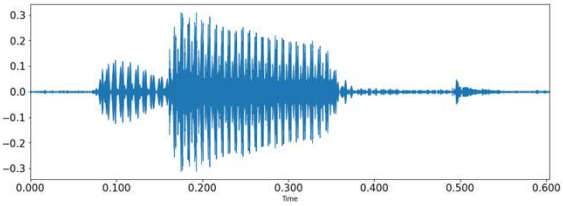


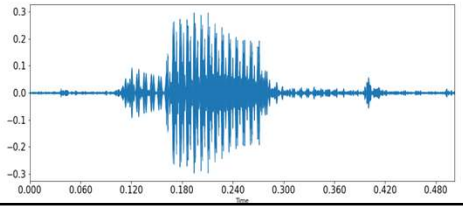
- The prediction clearly does not depend only on the last feature vector in the sequence but also on some feature vectors which appear before
- Network is performing the same task at each step (input: 2d-data point, output: character class)-Don't care about intermediate outputs

12

Sequence Learning Problem: Spoken Word Recognition

- Context: Limited vocabulary word recognition
  - Example: Systems working on the voice commands
- Each spoken word is a sequence of sounds
  - Example spoken word: "deep"
  - Sequence of sounds: /d/ /e/ /p/
- Duration of the utterances of the same word varies from each other

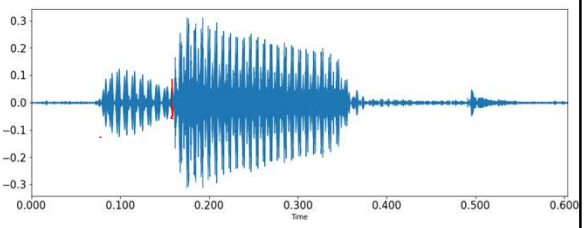




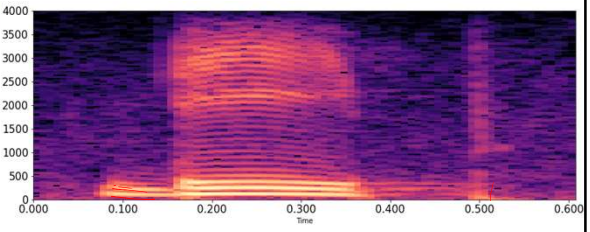
13

Sequence Learning Problem: Spoken Word Recognition

- Representing the speech in time and frequency domains:
- Time domain representation: Speech waveform (time vs amplitude)



- Frequency domain representation: Spectrogram (time vs frequency) – Time-Frequency representation of the speech signal



14

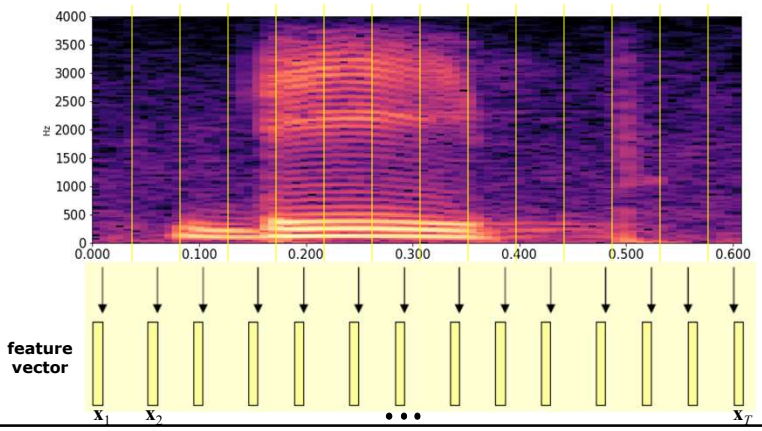
Sequence Learning Problem: Spoken Word Recognition

- Representing the speech in time and frequency domains:
  - Time domain representation: Speech waveform (time vs amplitude)
  - Frequency domain representation: Spectrogram (time vs frequency) - Time-Frequency representation of the speech signal
- Features are extracted from frequency domine representation
- The speech signal is a slowly time varying signal
- Quasi-stationary: When examined over a sufficiently short period of time (20-30 ms), its characteristics are fairly stationary
- Over long periods of time the signal characteristics change to reflect the different speech sounds being spoken

15

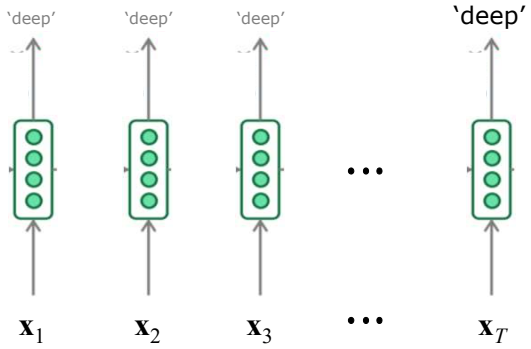
Representation of a Speech Signal for Speech Processing Tasks (Spoken Word Recognition)

- Speech signal represented as a sequence of d-dimensional feature vectors
  - Short time spectral analysis at every window of size 20ms to 30ms with the shift of 10ms to 15ms
  - A d-dimensional feature vector is extracted from each of the window



16

Sequence Learning Problem: Spoken Word Recognition

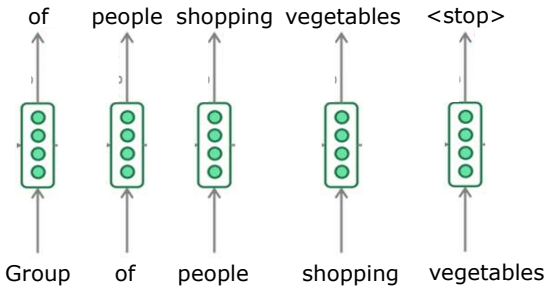


- The prediction clearly does not depend only on the last feature vector in the sequence but also on some feature vectors which appear before
- As the duration of the utterance varies for each example, the length of the sequence of feature vectors varies
- Network is performing the same task at each step (input: feature vector (speech window), output: spoken word (class))-Don't care about intermediate outputs

17

Sequence Learning Problem

- Natural sentence generation
  - Given $t - i$ words predict the t^{th} word
- Each input corresponding to one time step



- Inputs are dependent on each other
- Lengths of the inputs are not fixed (sentences could have arbitrary number of words)
- Each network is performing same task (input: word, output: word)
- **Sequence learning problem**:
  - Look at the sequence of (dependent) inputs and produce output(s)

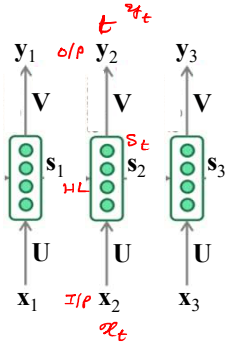
18

Neural Network Model for Sequence Learning Problem

- Model for dealing with sequences should
  - Account for dependence between inputs
  - Account for variable number of inputs
  - Make sure that the function executed at each time step is the same

19

Neural Network Model for Sequence Learning Problem



- Function executed at each time step:

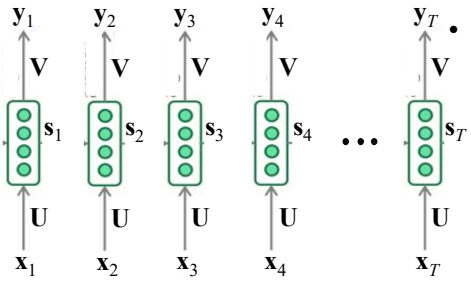
$$\mathbf{s}_t = \Omega(\mathbf{U}\mathbf{x}_t + \mathbf{b})$$
$$\mathbf{y}_t = \Phi(\mathbf{V}\mathbf{s}_t + \mathbf{c})$$

  - t is time step
  - $\mathbf{x}_t \in \mathbb{R}^d$, d is dimension of input variable
  - $\mathbf{y}_t \in \mathbb{R}^K$, K is dimension of output variable (e.g. K classes)
  - $\mathbf{s}_t \in \mathbb{R}^M$, M is number of nodes in hidden layer (last)

- Since the same function to be executed at each time step we should share the same network (i.e., same parameters at each time step)

20

Neural Network Model for Sequence Learning Problem



- Function executed at each time step:

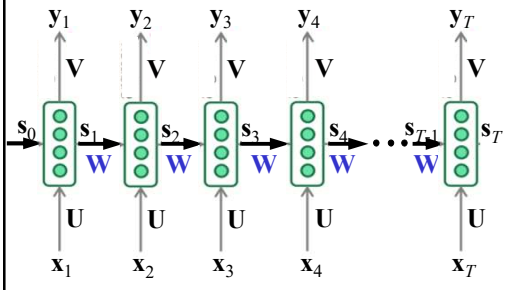
$$\mathbf{s}_t = \Omega(\mathbf{U}\mathbf{x}_t + \mathbf{b})$$
$$\mathbf{y}_t = \Phi(\mathbf{V}\mathbf{s}_t + \mathbf{c})$$

  - t is time step

- Since the same function to be executed at each time step we should share the same network (i.e., same parameters at each time step)
- This parameter sharing also ensures that the network becomes agnostic to the length (size) of the input
- Since same function (with same parameters) is computed at each time step, the number of timesteps doesn't matter
- Just create multiple copies of the network and execute them at each time step

21

Neural Network Model for Sequence Learning Problem



- How do you account for dependence between inputs?
- Add a recurrent connection in the network to account for dependence between inputs

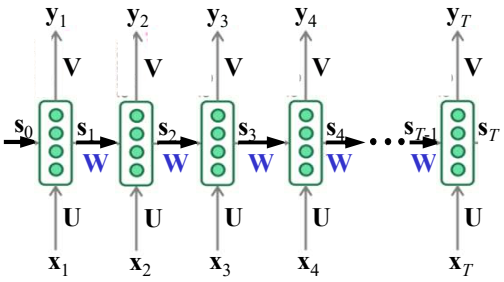
- Function executed at each time step:

$$\mathbf{s}_t = \Omega(\mathbf{U}\mathbf{x}_t + \mathbf{W}\mathbf{s}_{t-1} + \mathbf{b})$$
$$\mathbf{y}_t = \Phi(\mathbf{V}\mathbf{s}_t + \mathbf{c})$$

- $\mathbf{s}_t$ is the state of the network at time step t
  - $\mathbf{s}_0$ is constant
- The parameters $\mathbf{W}$, $\mathbf{U}$, $\mathbf{V}$, $\mathbf{c}$, $\mathbf{b}$ are shared across time steps

22

Recurrent Neural Network (RNN)

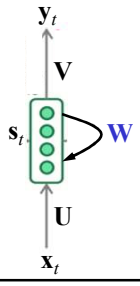


tanh.

$$\mathbf{s}_t = \Omega(\mathbf{U}\mathbf{x}_t + \mathbf{W}\mathbf{s}_{t-1} + \mathbf{b})$$

$$\mathbf{y}_t = \Phi(\mathbf{V}\mathbf{s}_t + \mathbf{c})$$

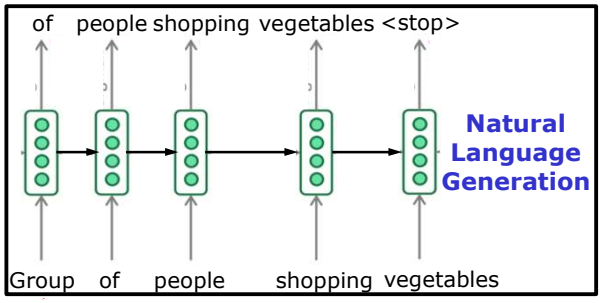
- More compactly represented as:



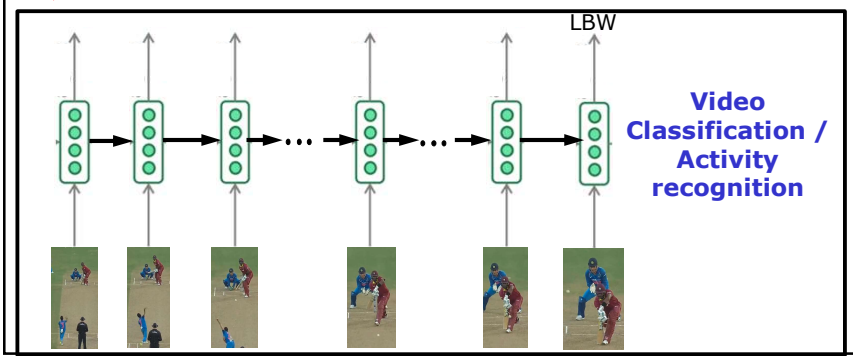
$$\mathbf{y}_t = f(\mathbf{x}_t, \mathbf{s}_{t-1}, \mathbf{U}, \mathbf{V}, \mathbf{W}, \mathbf{b}, \mathbf{c})$$

23

RNN for Sequence Learning Problem

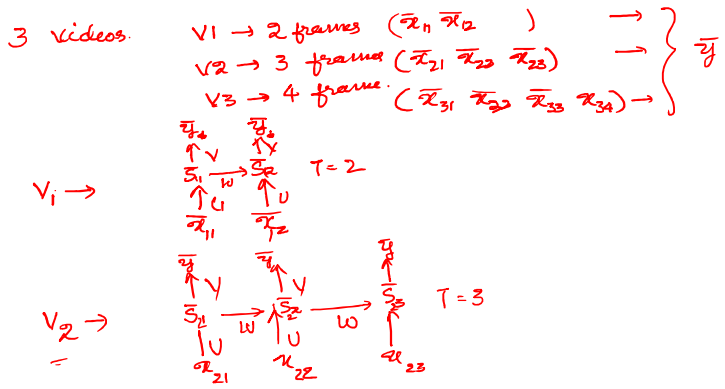


- Recurrent connections between time steps account for dependence between inputs



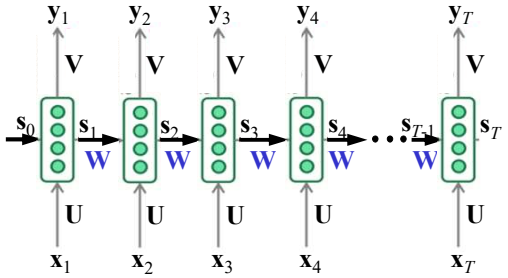
24

Training RNN



25

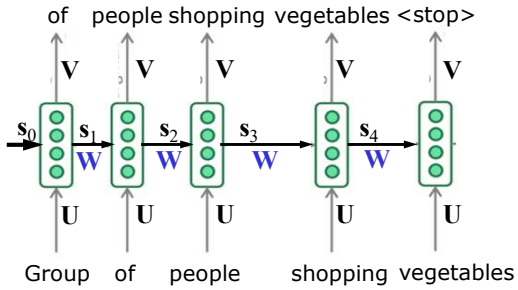
Training RNN: Backpropagation Through Time (BPTT)



- Gradient descent method
- Error backpropagation algorithm
- Forward computation:
  - Innerproduct computation over all the time steps
  - Activation function computation over all the time steps
- Backward operation:
  - Error/Loss calculation and propagation over all previous time steps
  - Modification of weights over all previous time steps

26

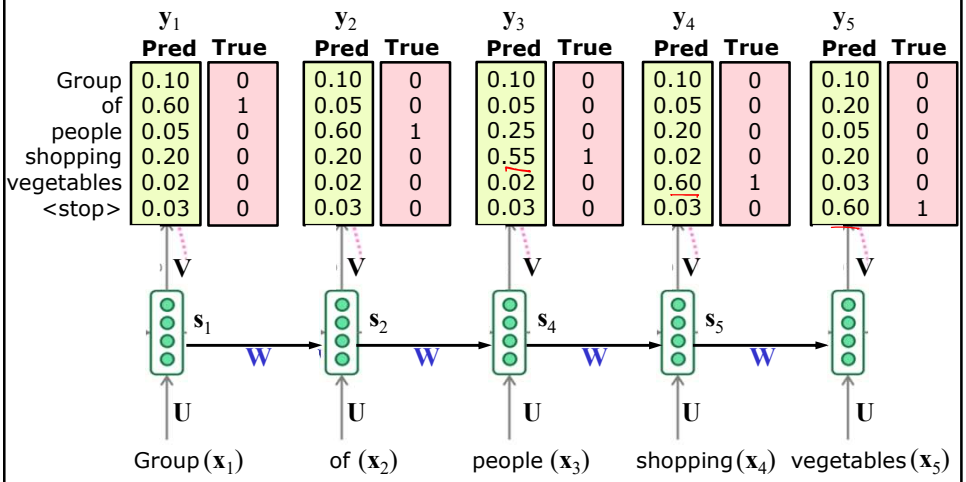
Backpropagation Through Time (BPTT)



- Consider the task of natural language generation (predicting the next word)
- For simplicity assume that there are only 6 words in the vocabulary (Group, of, people, shopping, vegetables, <stop>)
- At each time step we need to predict one of these 6 words
- Suitable output function: Softmax
- Suitable loss function: Cross Entropy

27

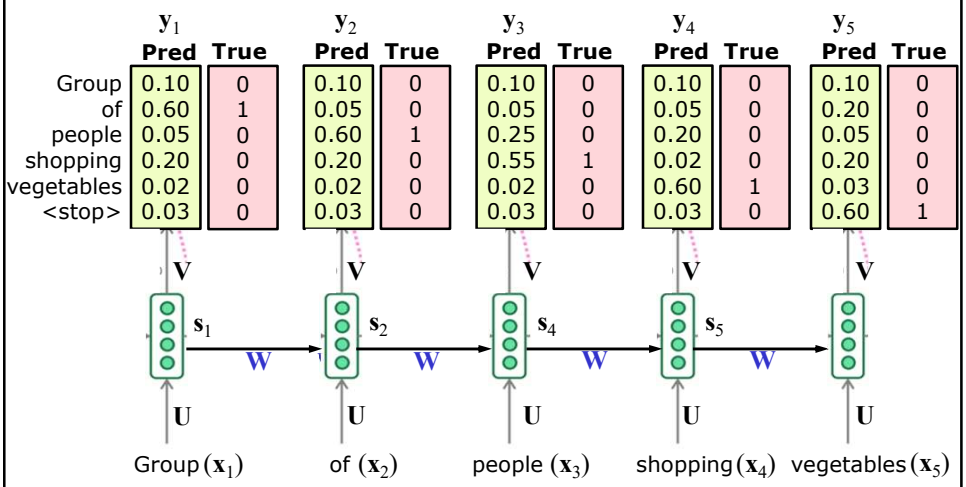
Backpropagation Through Time (BPTT)



- Initialize parameter set $\theta = \{U, V, W, c, b\}$ randomly and the network predicts the probabilities

28

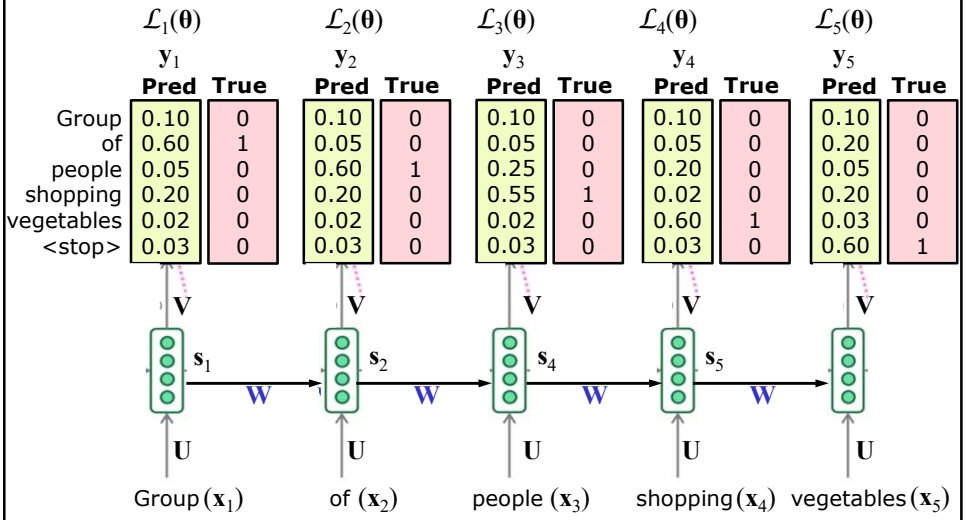
Backpropagation Through Time (BPTT)



- Need to answer following 2 questions:
  - What is the total loss made by the model?
  - How do we backpropagate this loss function and update the parameters of the network

29

Backpropagation Through Time (BPTT)



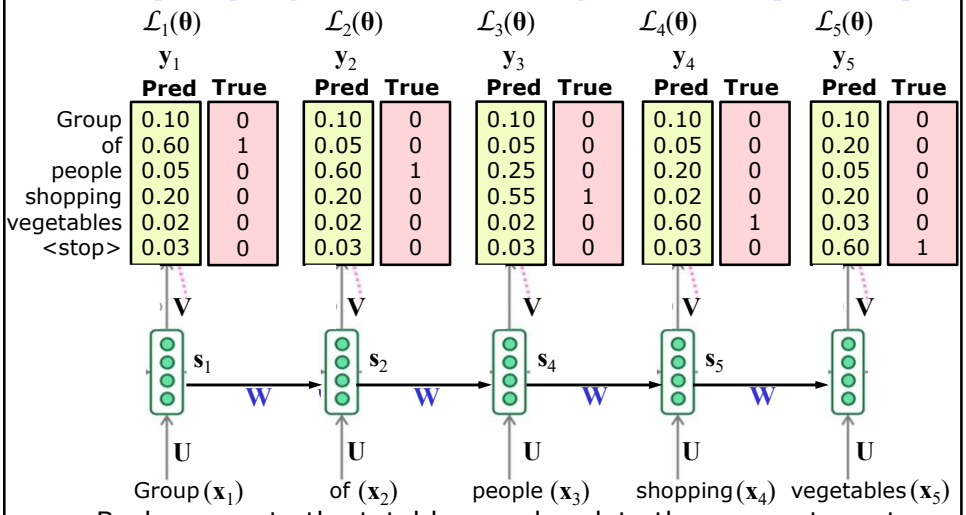
- Compute the loss over each time step: $\mathcal{L}_t(\theta)$
- Total loss is the sum of the loss over all time steps

$$\mathcal{L}(\theta) = \sum_{t=1}^{T} \mathcal{L}_t(\theta)$$

T: Number of time steps

30

Backpropagation Through Time (BPTT)



- Backpropagate the total loss and update the parameter set θ of the network
- For backpropagation we need to compute the gradients w.r.t. $\mathbf{U}$, $\mathbf{V}$, $\mathbf{W}$, $\mathbf{c}$, $\mathbf{b}$

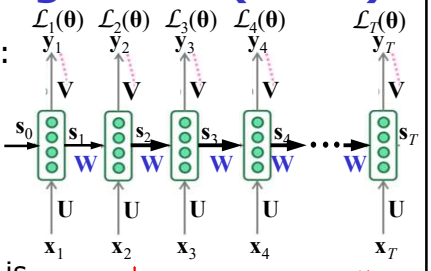
31

Backpropagation Through Time (BPTT)

- Change in the weight matrix $\mathbf{V}$:

$$\Delta \mathbf{V} = -\eta \nabla_{\mathbf{V}} \mathcal{L}(\boldsymbol{\theta})$$

$$\nabla_{\mathbf{V}} \mathcal{L}(\boldsymbol{\theta}) = \frac{\partial \mathcal{L}(\boldsymbol{\theta})}{\partial \mathbf{V}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t(\boldsymbol{\theta})}{\partial \mathbf{V}}$$

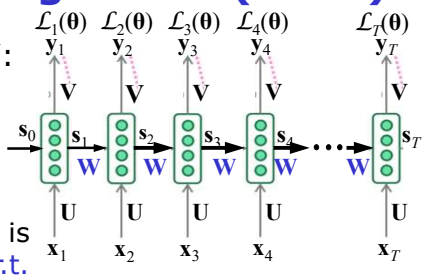


  - Each term in the summation is simply the derivative of the loss w.r.t. the weights in the output layer
  - This gradient computation is same as that of the backpropagation in fully connected neural network (between hidden and output layer)
- Updating the weight $\mathbf{V}$:

$$\mathbf{V} = \mathbf{V} + \Delta \mathbf{V}$$

32

Backpropagation Through Time (BPTT)



- Change in the weight matrix $\mathbf{U}$:

$$\Delta\mathbf{U} = -\eta\nabla_u \mathcal{L}(\boldsymbol{\theta})$$

$$\nabla_u \mathcal{L}(\boldsymbol{\theta}) = \frac{\partial \mathcal{L}(\boldsymbol{\theta})}{\partial \mathbf{U}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t(\boldsymbol{\theta})}{\partial \mathbf{U}}$$

  - Each term is the summation is the derivative of the loss w.r.t. the weights in the input/hidden layer
  - This gradient computation is same as that of the backpropagation in fully connected neural network (between hidden layers and hidden layer to input layer)
  - By chain rule, assuming single hidden layer

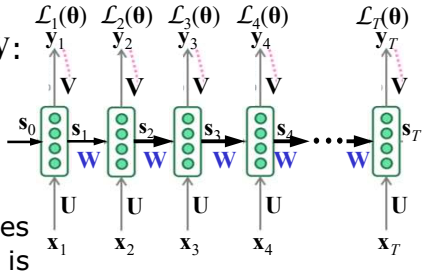
$$\frac{\partial \mathcal{L}_t(\boldsymbol{\theta})}{\partial \mathbf{U}} = \frac{\partial \mathcal{L}_t(\boldsymbol{\theta})}{\partial \mathbf{y}_t}\frac{\partial \mathbf{y}_t}{\partial \mathbf{a}_{2t}}\frac{\partial \mathbf{a}_{2t}}{\partial \mathbf{s}_t}\frac{\partial \mathbf{s}_t}{\partial \mathbf{a}_{1t}}\frac{\partial \mathbf{a}_{1t}}{\partial \mathbf{U}} \qquad \mathbf{y}_t = \Phi(\mathbf{a}_{2t}) \quad \mathbf{a}_{2t} = \mathbf{V}\mathbf{s}_t + \mathbf{c}$$

$$\mathbf{s}_t = \Omega(\mathbf{a}_{1t}) \quad \mathbf{a}_{1t} = \mathbf{U}\mathbf{x}_t + \mathbf{W}\mathbf{s}_{t-1} + \mathbf{b}$$

- Updating the weight $\mathbf{U}$: $\mathbf{U} = \mathbf{U} + \Delta\mathbf{U}$

33

Backpropagation Through Time (BPTT)



- Change in the weight matrix $\mathbf{W}$:

$$\Delta\mathbf{W} = -\eta\nabla_w \mathcal{L}(\boldsymbol{\theta})$$

$$\nabla_w \mathcal{L}(\boldsymbol{\theta}) = \frac{\partial \mathcal{L}(\boldsymbol{\theta})}{\partial \mathbf{W}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t(\boldsymbol{\theta})}{\partial \mathbf{W}}$$

  - By the chain rule of derivatives this gradient at time sept t is obtained by summing gradients along all the paths from $\mathcal{L}_t(\boldsymbol{\theta})$ to $\mathbf{W}$
  - Paths connecting from $\mathcal{L}_t(\boldsymbol{\theta})$ to $\mathbf{W}$:
    - $\mathcal{L}_t(\boldsymbol{\theta})$ depends on $\mathbf{s}_t$
    - $\mathbf{s}_t$ in turn depends on $\mathbf{s}_{t-1}$ and $\mathbf{W}$
    - $\mathbf{s}_{t-1}$ in turn depends on $\mathbf{s}_{t-2}$ and $\mathbf{W}$
    - ...
    - $\mathbf{s}_2$ in turn depends on $\mathbf{s}_1$ and $\mathbf{W}$
    - $\mathbf{s}_1$ in turn depends on $\mathbf{s}_0$ and $\mathbf{W}$ ($\mathbf{s}_0$ is constant starting state)

Ordered network

34

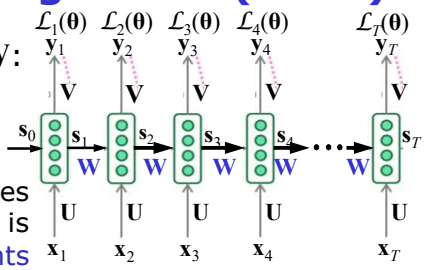
Backpropagation Through Time (BPTT)

- Change in the weight matrix $\mathbf{W}$:

$$\nabla_w \mathcal{L}(\boldsymbol{\theta}) = \frac{\partial \mathcal{L}(\boldsymbol{\theta})}{\partial \mathbf{W}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t(\boldsymbol{\theta})}{\partial \mathbf{W}}$$

  - By the chain rule of derivatives this gradient at time sept t is obtained by summing gradients along all the paths from $\mathcal{L}_t(\boldsymbol{\theta})$ to $\mathbf{W}$

$$\frac{\partial \mathcal{L}_t(\boldsymbol{\theta})}{\partial \mathbf{W}} = \frac{\partial \mathcal{L}_t(\boldsymbol{\theta})}{\partial \mathbf{s}_t} \frac{\partial \mathbf{s}_t}{\partial \mathbf{W}}$$



$$\mathbf{s}_t = \Omega(\mathbf{U}\mathbf{x}_t + \mathbf{W}\mathbf{s}_{t-1} + \mathbf{b})$$

35

Backpropagation Through Time (BPTT)

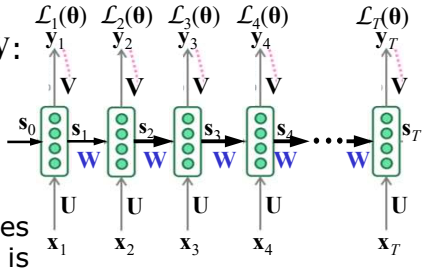
- Change in the weight matrix $\mathbf{W}$:

$$\Delta \mathbf{W} = -\eta \nabla_w \mathcal{L}(\boldsymbol{\theta})$$

$$\nabla_w \mathcal{L}(\boldsymbol{\theta}) = \frac{\partial \mathcal{L}(\boldsymbol{\theta})}{\partial \mathbf{W}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t(\boldsymbol{\theta})}{\partial \mathbf{W}}$$

  - By the chain rule of derivatives this gradient at time sept t is obtained by summing gradients along all the paths from $\mathcal{L}_t(\boldsymbol{\theta})$ to $\mathbf{W}$

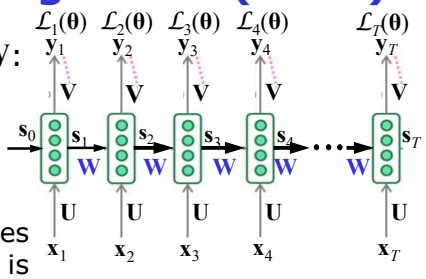
$$\frac{\partial \mathcal{L}_t(\boldsymbol{\theta})}{\partial \mathbf{W}} = \frac{\partial \mathcal{L}_t(\boldsymbol{\theta})}{\partial \mathbf{s}_t} \left(\sum_{k=1}^{t} \frac{\partial \mathbf{s}_t}{\partial \mathbf{s}_k} \frac{\partial^+ \mathbf{s}_k}{\partial \mathbf{W}} \right)$$



$$\mathbf{s}_t = \Omega(\mathbf{U}\mathbf{x}_t + \mathbf{W}\mathbf{s}_{t-1} + \mathbf{b})$$

36

Backpropagation Through Time (BPTT)



- Change in the weight matrix $\mathbf{W}$:

$$\Delta\mathbf{W} = -\eta\nabla_w\mathcal{L}(\boldsymbol{\theta})$$

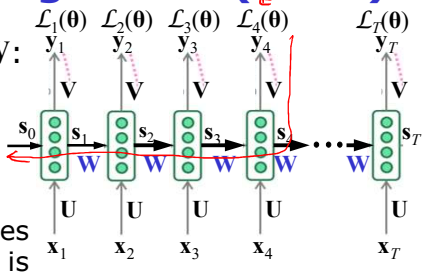
$$\nabla_w\mathcal{L}(\boldsymbol{\theta}) = \frac{\partial\mathcal{L}(\boldsymbol{\theta})}{\partial\mathbf{W}} = \sum_{t=1}^{T}\frac{\partial\mathcal{L}_t(\boldsymbol{\theta})}{\partial\mathbf{W}}$$

  - By the chain rule of derivatives this gradient at time sept t is obtained by summing gradients along all the paths from $\mathcal{L}_t(\boldsymbol{\theta})$ to $\mathbf{W}$

$$\nabla_w\mathcal{L}(\boldsymbol{\theta}) = \frac{\partial\mathcal{L}(\boldsymbol{\theta})}{\partial\mathbf{W}} = \sum_{t=1}^{T}\frac{\partial\mathcal{L}_t(\boldsymbol{\theta})}{\partial\mathbf{s}_t}\left(\sum_{k=1}^{t}\frac{\partial\mathbf{s}_t}{\partial\mathbf{s}_k}\frac{\partial^+\mathbf{s}_k}{\partial\mathbf{W}}\right)$$

$\frac{\partial^+\mathbf{s}_k}{\partial\mathbf{W}}$ is a tensor

Backpropagation Through Time (BPTT)



- Change in the weight matrix $\mathbf{W}$:

$$\Delta\mathbf{W} = -\eta\nabla_w\mathcal{L}(\boldsymbol{\theta})$$

$$\nabla_w\mathcal{L}(\boldsymbol{\theta}) = \frac{\partial\mathcal{L}(\boldsymbol{\theta})}{\partial\mathbf{W}} = \sum_{t=1}^{T}\frac{\partial\mathcal{L}_t(\boldsymbol{\theta})}{\partial\mathbf{W}}$$

  - By the chain rule of derivatives this gradient at time sept t is obtained by summing gradients along all the paths from $\mathcal{L}_t(\boldsymbol{\theta})$ to $\mathbf{W}$

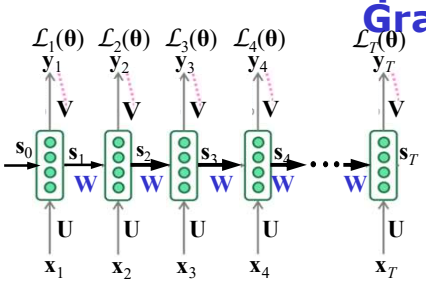
$$\nabla_w\mathcal{L}(\boldsymbol{\theta}) = \frac{\partial\mathcal{L}(\boldsymbol{\theta})}{\partial\mathbf{W}} = \sum_{t=1}^{T}\frac{\partial\mathcal{L}_t(\boldsymbol{\theta})}{\partial\mathbf{s}_t}\left(\sum_{k=1}^{t}\frac{\partial\mathbf{s}_t}{\partial\mathbf{s}_k}\frac{\partial^+\mathbf{s}_k}{\partial\mathbf{W}}\right)$$

$\frac{\partial^+\mathbf{s}_k}{\partial\mathbf{W}}$ is a tensor

- Updating the weight $\mathbf{W}$: Backpropagate over all previous time steps

$$\mathbf{W} = \mathbf{W} + \Delta\mathbf{W}$$

Problem of Exploding and Vanishing Gradients



- Consider the gradient with respect to recurrent connection weights

$$\nabla_w \mathcal{L}(\theta) = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{W}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t(\theta)}{\partial \mathbf{W}}$$

$$\frac{\partial \mathcal{L}_t(\theta)}{\partial \mathbf{W}} = \frac{\partial \mathcal{L}_t(\theta)}{\partial \mathbf{s}_t}\left(\sum_{k=1}^{t} \frac{\partial \mathbf{s}_t}{\partial \mathbf{s}_k}\frac{\partial^+ \mathbf{s}_k}{\partial \mathbf{W}}\right)$$

$$\mathbf{s}_j = \Omega(\mathbf{a}_j) \qquad \mathbf{a}_j = \mathbf{U}\mathbf{x}_j + \mathbf{W}\mathbf{s}_{j-1} + b$$

$$\frac{\partial \mathbf{s}_j}{\partial \mathbf{s}_{j-1}} = \frac{\partial \mathbf{s}_j}{\partial \mathbf{a}_j}\frac{\partial \mathbf{a}_j}{\partial \mathbf{s}_{j-1}}$$

$$\frac{\partial \mathbf{s}_j}{\partial \mathbf{s}_{j-1}} = \frac{\partial \Omega(\mathbf{a}_j)}{\partial \mathbf{a}_j}\frac{\partial \mathbf{a}_j}{\partial \mathbf{s}_{j-1}}$$

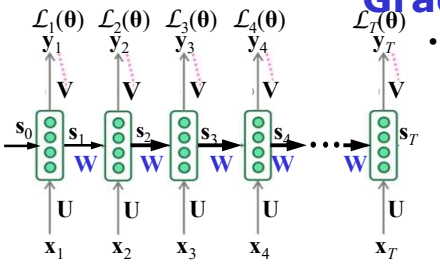
$$\frac{\partial \mathbf{s}_t}{\partial \mathbf{s}_k} = \frac{\partial \mathbf{s}_t}{\partial \mathbf{s}_{t-1}}\frac{\partial \mathbf{s}_{t-1}}{\partial \mathbf{s}_{t-2}}\cdots\frac{\partial \mathbf{s}_{k+1}}{\partial \mathbf{s}_k} = \prod_{j=k+1}^{t}\frac{\partial \mathbf{s}_j}{\partial \mathbf{s}_{j-1}}$$

$$\frac{\partial \mathbf{s}_t}{\partial \mathbf{s}_k} = \prod_{j=k+1}^{t}\frac{\partial \Omega(\mathbf{a}_j)}{\partial \mathbf{a}_j}\frac{\partial \mathbf{a}_j}{\partial \mathbf{s}_{j-1}}$$

$$\frac{\partial \mathcal{L}_t(\theta)}{\partial \mathbf{W}} = \frac{\partial \mathcal{L}_t(\theta)}{\partial \mathbf{s}_t}\left(\sum_{k=1}^{t}\left[\prod_{j=k+1}^{t}\frac{\partial \Omega(\mathbf{a}_j)}{\partial \mathbf{a}_j}\frac{\partial \mathbf{a}_j}{\partial \mathbf{s}_{j-1}}\right]\frac{\partial^+ \mathbf{s}_k}{\partial \mathbf{W}}\right)$$

39

Problem of Exploding and Vanishing Gradients



- Consider the gradient with respect to recurrent connection weights

$$\nabla_w \mathcal{L}(\theta) = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{W}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t(\theta)}{\partial \mathbf{W}}$$

$$\frac{\partial \mathcal{L}_t(\theta)}{\partial \mathbf{W}} = \frac{\partial \mathcal{L}_t(\theta)}{\partial \mathbf{s}_t}\left(\sum_{k=1}^{t} \frac{\partial \mathbf{s}_t}{\partial \mathbf{s}_k}\frac{\partial^+ \mathbf{s}_k}{\partial \mathbf{W}}\right)$$

$$\frac{\partial \mathcal{L}_t(\theta)}{\partial \mathbf{W}} = \frac{\partial \mathcal{L}_t(\theta)}{\partial \mathbf{s}_t}\left(\sum_{k=1}^{t}\left[\prod_{j=k+1}^{t}\frac{\partial \Omega(\mathbf{a}_j)}{\partial \mathbf{a}_j}\frac{\partial \mathbf{a}_j}{\partial \mathbf{s}_{j-1}}\right]\frac{\partial^+ \mathbf{s}_k}{\partial \mathbf{W}}\right)$$

If $\Omega(a) \Rightarrow$ logistic $\rightarrow$ range is bounded b/t 0 & 1

$\Omega'(a) = \Omega(a)(1-\Omega(a)) \rightarrow$ range bounded b/t 0 to 0.25

$a = \pm\infty \rightarrow \Omega'(a) = 0$

$a = 0 \rightarrow \Omega'(a) = (0.5)(1-0.5) = 0.25$

$\Omega(a) \rightarrow$ tanh $\rightarrow$ -1 to $+1$

$\Omega'(a) \rightarrow 1 - \Omega^2(a) \rightarrow 0$ to 1

$t \approx T \quad k = 1$

$\frac{\partial \mathbf{s}_t}{\partial \mathbf{s}_k} \rightarrow 0$

$\frac{\partial \mathcal{L}_t(\theta)}{\partial \mathbf{W}} \rightarrow 0$

40

Problem of Exploding and Vanishing Gradients

- One simple way of avoiding this is to use truncated backpropogation where we restrict the product to less than $t-k$ terms
- Other ways to avoid exploding and vanishing gradient is by using LSTM and GRU

41

Summery: RNN

- RNN used for sequential learning problem
  - Each input is dependent on the previous or future input
  - In many applications the input is not of a fixed size
- RNN uses backpropagation through time for training
- RNN may suffer from vanishing/exploding gradients problem
- This issue is addressed in long short term memory (LSTM) model
  - LSTM is a special case of RNN

42

Text Books

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, Deep learning, MIT Press, Available online: http://www.deeplearningbook.org, 2016
2. Charu C. Aggarwal, Neural Networks and Deep Learning, Springer, 2018
3. B. Yegnanarayana, Artificial Neural Networks, Prentice-Hall of India, 1999.
4. Satish Kumar, Neural Networks - A Class Room Approach, Second Edition, Tata McGraw-Hill, 2013.
5. S. Haykin, Neural Networks and Learning Machines, Prentice Hall of India, 2010.
6. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
7. J. Han and M. Kamber, *Data Mining: Concepts and Techniques*, Third Edition, Morgan Kaufmann Publishers, 2011.
8. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, Academic Press, 2009.

43